

20 Essential Pandas Methods

with Real-Life Use Cases

1. read_csv()

Reads a CSV file and loads it into a DataFrame.

```
df = pd.read_csv('sales_data.csv')
```

Real Life: Loading customer purchase records or employee data from CSV files for analysis.

2. head()

Returns the first N rows of the DataFrame (default 5).

```
df.head(10)
```

Real Life: Quickly previewing top rows of a large dataset after import.

3. tail()

Returns the last N rows of the DataFrame.

```
df.tail(5)
```

Real Life: Checking the most recent transactions in a financial log.

4. info()

Displays a summary of the DataFrame including data types and null counts.

```
df.info()
```

Real Life: Understanding data quality issues and column types before cleaning.

5. describe()

Generates descriptive statistics for numerical columns.

```
df.describe()
```

Real Life: Getting mean salary, max age, and std deviation of sales figures.

6. dropna()

Removes rows (or columns) with missing values.

```
df.dropna(subset=['email'])
```

Real Life: Cleaning a customer database by removing rows with missing email fields.

7. fillna()

Fills missing values with a specified value or method.

```
df['age'].fillna(df['age'].mean())
```

Real Life: Filling missing patient ages with the average age in a medical dataset.

8. groupby()

Groups data by one or more columns for aggregation.

```
df.groupby('department')['salary'].mean()
```

Real Life: Calculating average salary per department in an HR dataset.

9. merge()

Merges two DataFrames like a SQL JOIN operation.

```
pd.merge(orders_df, customers_df, on='customer_id')
```

Real Life: Joining order records with customer details using a shared customer ID.

10. sort_values()

Sorts the DataFrame by one or more columns.

```
df.sort_values('revenue', ascending=False)
```

Real Life: Ranking products by highest to lowest revenue in a sales report.

11. value_counts()

Counts the frequency of unique values in a column.

```
df['city'].value_counts()
```

Real Life: Finding which cities have the most customers or orders.

12. apply()

Applies a function along an axis of the DataFrame.

```
df['name'].apply(lambda x: x.upper())
```

Real Life: Standardizing all product names to uppercase in an inventory sheet.

13. rename()

Renames columns or index labels.

```
df.rename(columns={'emp_id': 'Employee ID'})
```

Real Life: Renaming technical column names to human-readable labels for a report.

14. drop()

Removes specified rows or columns from the DataFrame.

```
df.drop(columns=['temp_col', 'unused'])
```

Real Life: Removing redundant or sensitive columns before sharing a dataset.

15. loc[]

Accesses rows and columns by label-based indexing.

```
df.loc[df['score'] > 90, ['name', 'score']]
```

Real Life: Extracting names and scores of students who scored above 90%.

16. iloc[]

Accesses rows and columns by integer-based positional indexing.

```
df.iloc[0:5, 1:4]
```

Real Life: Extracting a specific block of rows and columns from a large dataset.

17. pivot_table()

Creates a pivot table similar to Excel's pivot functionality.

```
pd.pivot_table(df, values='sales', index='region', columns='month')
```

Real Life: Summarising monthly sales figures by region for a business dashboard.

18. to_csv()

Exports the DataFrame to a CSV file.

```
df.to_csv('cleaned_data.csv', index=False)
```

Real Life: Saving the processed/cleaned dataset to share with a team member.

19. isnull()

Returns a boolean DataFrame indicating missing values.

```
df.isnull().sum()
```

Real Life: Counting missing entries per column to assess data completeness.

20. corr()

Computes pairwise correlation between numerical columns.

```
df.corr()
```

Real Life: Analysing relationship between age, income and spending score in a marketing dataset.